

Customer Churn Prediction and Retention Strategy

Siddhi Mhasawade
Dept. of Computer Science
Purdue University Northwest
Hammond, USA
smhasawa@pnw.edu

Abstract— This project focuses on building a fast and reliable system to predict customer churn. Instead of relying on traditional “black-box” models, it combines a MySQL- based data pipeline with explainable AI tools like SHAP and LIME to make the results more transparent and easier to understand.

Keywords— Customer Churn, Explainable AI, SHAP, LIME, MySQL, Logistic Regression, Black Box, System Performance.

I. INTRODUCTION

Customer churn, also known as customer attrition. It refers to loss of customers when they stop using a company’s services or cancel their subscriptions. In the telecommunications industry, churn is especially important because acquiring new customers typically costs much more than retaining existing ones.

Even though churn has been studied for a long time, it remains a significant challenge in practice. Machine learning models can predict which customers might leave, but they often don’t clearly explain why. That lack of explanation makes it harder to use the results in actual business decisions.

To address this gap, this report presents an end-to-end system that combines relational databases with interpretable machine learning models and explainable AI. The focus is not just on predicting churn, but also on understanding what is causing it, so businesses can respond in a more targeted way.

Customer churn is commonly measured using the formula:

$$\text{Churn Rate} = (\text{Customers Lost} / \text{Total Customers}) \times 100$$

II. PROBLEM STATEMENT

One of the main challenges in customer churn prediction is the “black box” models. Many ensemble models such as XGBoost, Random Forest achieve high accuracy but they often become black boxes, that are hard for humans to understand. While they can accurately identify customers who are likely to leave, they don’t clearly explain why, making it harder for businesses to take meaningful action.

Another challenge was making sure the system could smoothly handle real-time data from MySQL while also running heavier explanation methods like SHAP and LIME. Without proper optimisation, this could slow down the overall performance and increase resource usage.

This project focuses on solving these issues by creating a churn prediction system that is not only accurate, but also transparent, efficient, and easy to use.

III. PROPOSED METHODOLOGY

A. Data Sourcing and Feature Engineering

The primary data source for this project was a relational database managed using MySQL. The dataset was first

imported into MySQL and stored in the *customers_train* table, where SQL exploration revealed a churn baseline of 26.54%. I chose to perform the heavy lifting of data transformation directly within MySQL Workbench to optimise efficiency.

Using SQL CASE statements, I translated raw tenure months into meaningful life-cycle categories and engineered a HighChargeFlag to highlight customers with high-risk billing patterns. By merging contract loyalty with tenure data at the database level, I was able to feed a refined, high-signal dataset into the Python environment.

B. Model Development and XAI Integration

After completing the ETL process, the cleaned dataset was loaded into Python to build and test the model. The data was fetched directly from a local MySQL database using *mysql.connector* along with SQL queries.

To ensure the model treated all numerical values fairly, StandardScaler was applied to the dataset. Since features like MonthlyCharges have a much larger range than binary flags, scaling them was critical to prevent any single variable from unfairly dominating the Logistic Regression model’s decision-making.

To ensure model integrity and avoid data leakage, the project used two completely separate datasets stored in different MySQL tables. The *customers_train* table was used to train and fit the Logistic Regression model along with the scaling process. A separate *customers_test* table was then used only for the Streamlit “Randomize Customer” feature. This setup ensures that all predictions and explanations shown in the interface come from unseen data, making the results more realistic and closer to how the system would behave in real-world scenarios.

IV. SYSTEM ARCHITECTURE

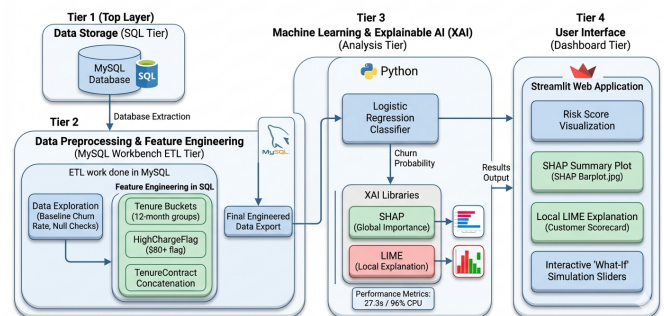


Fig.1 System Architecture showing the integration between the MySQL relational database and the Python-based machine learning pipeline.

V. EXPERIMENTAL SETUP AND EVALUATION

A. Hardware and Environment Specifications

The system was developed and tested on an Apple M2 architecture with 8GB of unified memory. The backend was powered by a local MySQL 8.0 instance, while the frontend and predictive logic were implemented using Python 3.9 and Streamlit. Data ingestion was handled via the `mysql.connector` library, ensuring low-latency communication between the relational database and the machine learning pipeline.

B. Performance Metrics

The system's efficiency was benchmarked by tracking execution latency and resource consumption across the full pipeline.

Parameter	Measurement
Total Pipeline Latency	27.34 Seconds
Peak CPU Utilisation	96.0%
Steady-State Memory	185 MB
Database Retrieval Time	<1.0 Second

Table I. System Resource and Latency Benchmarks

C. Model Interpretability and Findings

The Logistic Regression model was evaluated using explainable AI techniques to better understand its behaviour. SHAP summary plots were used to examine overall feature importance, showing that factors such as contract type and monthly charges had the strongest influence on churn. In addition, LIME was applied to explain individual predictions by highlighting the key reasons behind a customer's churn risk. These explanations helped confirm that the model's decisions were consistent with expected business patterns.

VI. SYSTEM FLOWCHART

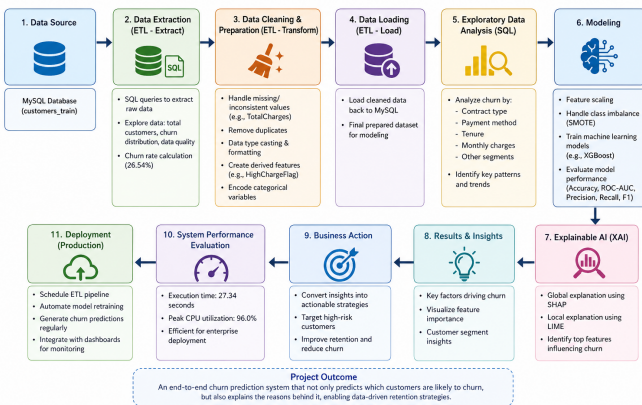


Fig 2. The flowchart depicts the actual flow of the system.

VII. KEY TECHNICAL CHALLENGES

During development of the system, a few important technical issues had to be resolved such as:

Database constraints: Initial ETL operations were restricted by MySQL's `SQL_SAFE_UPDATES` mode, which

blocked bulk updates. This was fixed by adjusting session settings to allow smoother data cleaning and feature engineering.

High computation load: Generating SHAP explanations for the dataset resulted in a peak CPU utilisation of 96.0%. To manage this, the background sample size was reduced to keep the system responsive without losing too much interpretability.

Feature consistency issues: One of the key challenges was making sure the SQL-based feature engineering aligned correctly with the encoding and transformations done later in Python using Scikit-learn.

VIII. TESTING

To make sure the whole system worked smoothly, it was tested in multiple stages:

Functional testing: The "Randomise Customer" feature was run around 50 times to check whether the sql query correctly pulled different customer records each time without duplicates or system errors.

Integration testing: Data coming from MySQL (such as tenure as integers) was verified to ensure it was correctly processed by the Scikit-learn pipeline without runtime errors.

User Interface (UI) testing: The Streamlit dashboard was tested for responsiveness. Even though SHAP computations caused high CPU usage at times, the interface remained stable and successfully displayed the results and visual explanations once processing was completed.

IX. RESULTS

A. Model Performance and Comparison

Four different Machine learning models were tested to determine which provided the best balance of accuracy and interpretability. As shown in Table II, the Logistic Regression model emerged as the top performer for this specific dataset.

Model	Accuracy	F-1 Score
Logistic Regression	75.99%	0.61
Decision Tree	74.20%	0.55
XGBoost	74.06%	0.53
Random Forest	72.07%	0.50

Table II. Shows that Logistic Regression model emerged as the top performer for this specific dataset.

B. Confusion Matrix Analysis

The confusion matrix (Fig. 3) shows that the model successfully identifies most loyal customers (4,034) and a strong number of churners (1,318). It does miss some churn cases (551) and produces some false alarms (1,140), but these are acceptable trade-offs given the benefit of catching over 1,300 at-risk customers for retention. Overall, the model shows a good balance between precision and recall, with an accuracy of 75.99%.

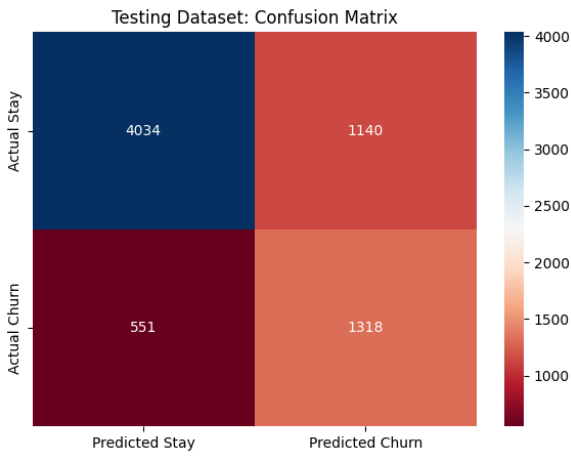


Fig 3. Confusion Matrix

C. Explainable AI Results

The SHAP graph provides an overall view of the model's behavior by showing which features, such as contract type and monthly charges, have the greatest impact on churn predictions across the entire dataset.

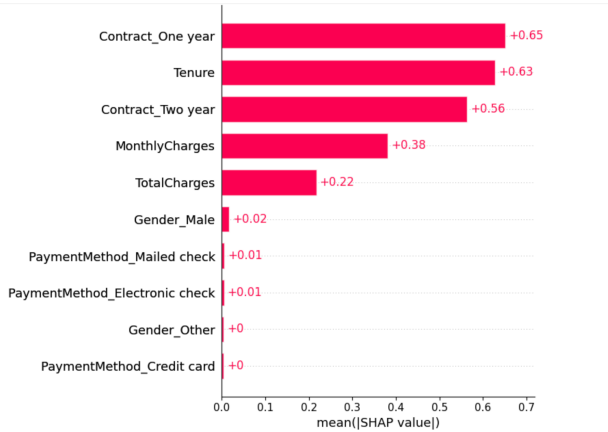


Fig 4. SHAP Results

The LIME graphs explain the prediction for an individual customer by highlighting the specific factors that influenced the model's decision.

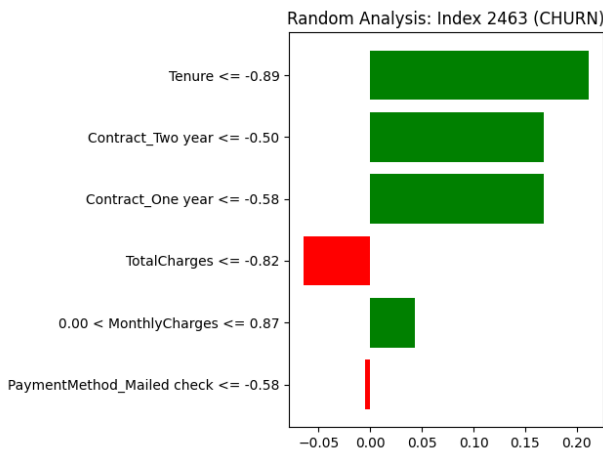


Fig 5. LIME results for customer index 2463, the model predicts CHURN

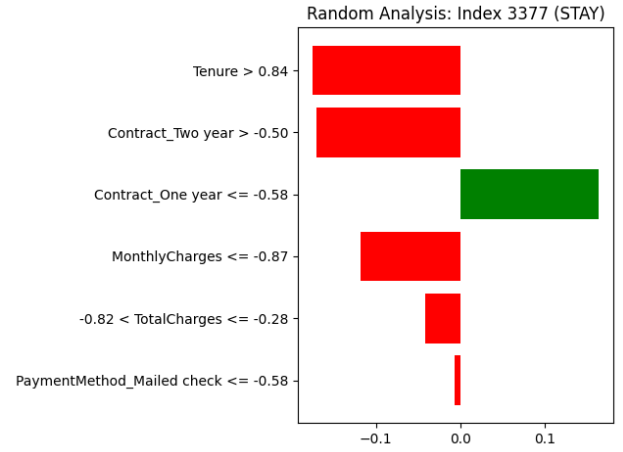


Fig 6. LIME results for customer index 3377, the model predicts STAY

X.

CONCLUSION

This project shows that combining a relational database with Explainable AI can create a churn prediction system that is both efficient and easy to understand. Handling feature engineering in MySQL made the preprocessing stage smoother and helped keep memory usage low.

At the same time, SHAP and LIME helped make the predictions easier to follow by showing the reasons behind each result. This way, the system does more than just identify customers who may churn. The system also gives practical insights that can help businesses take smarter retention steps.

This makes the system more useful for real-world business applications, as it allows decision-makers to understand customer behaviour better and take more focused retention actions. Overall, this project demonstrates how combining database optimisation with Explainable AI and interpretable machine learning can create a churn prediction framework that is efficient, transparent, and reliable for real-world business applications.

XI.

FUTURE WORK

1. Automated SQL Pipelines: Currently, the ETL process is manual in MySQL Workbench. Future versions could use Automated Triggers so that as soon as a customer's bill changes in the database, the Churn Risk score updates instantly on the dashboard without human intervention.

2. Sentiment Analysis Integration: We could link the database to Customer Service Logs. By using text analysis on support tickets, the model could "listen" for frustrated language, adding a "Sentiment Score" to the prediction for even higher accuracy.

3. Prescriptive Analytics: Moving beyond "Who will leave," the next step is "What should we do?" We could build a recommendation engine that suggests the exact discount or service upgrade most likely to keep a specific customer based on their LIME profile.

XII.

REFERENCES

[1] S. M. Lundberg and S. I. Lee, "A Unified Approach to Interpreting Model Predictions," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017. [Online] Available: <https://arxiv.org/abs/1705.07874>

[2] M. T. Ribeiro, S. Singh, and C. Guestrin, " 'Why Should I Trust You?': Explaining the Predictions of Any Classifier," *Proc. of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144, 2016. [Online] Available: <https://arxiv.org/abs/1602.04938>

[3] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online] Available: <https://jmlr.csail.mit.edu/papers/v12/pedregosa11a.html>

[4] A. C. Muller and S. Guido, Introduction to Machine Learning with Python: A Guide for Data Scientists. O'Reilly Media, Inc., 2016. [Online] Available: <https://www.oreilly.com/library/view/introduction-to-machine/9781449369880/>

[5] Streamlit Development Team, "Streamlit: The fastest way to build and share data apps," [Online] Available: <https://streamlit.io>.

[6] Oracle, "MySQL 8.0 Reference Manual," [Online] Available: <https://dev.mysql.com/doc/refman/8.0/en/>